

需求与版本管理系统

独立部署版 V1.5 · 完整开发基线

FastAPI / PostgreSQL / Redis / S3 Compatible Object Storage / Vue 3 / Element Plus

项目项	本版结论
部署形态	独立部署；不依赖现有后台账号、菜单、Cookie、权限或数据库
核心链路	Feedback → Requirement → Version → Release
后端技术	Python 3.12+ / FastAPI / SQLAlchemy 2 / Alembic
前端技术	Vue 3 / TypeScript / Vite / Element Plus
数据与中间件	PostgreSQL / Redis / S3 Compatible Object Storage
协同策略	revision 乐观锁 + Redis 编辑提示 + 409 冲突提示
终端策略	一套前端工程，同时适配 PC / 平板 / 手机
明确不做	单需求工时、Story Point、开发时长量化

文档版本：V1.5 日期：2026-09-20

文档目录

1. 本版目标与冻结范围
 2. 最终技术架构与部署拓扑
 3. 项目仓库与模块边界
 4. 后端完整开发基线
 5. 数据库与 Alembic 迁移基线
 6. API 契约与权限约束
 7. Feedback → Requirement → Version → Release 主链路
 8. 多人编辑与并发冲突
 9. 前端工程与 PC / Mobile 响应式实现
 10. 登录、RBAC 与系统管理
 11. 附件、通知、审计与安全
 12. 测试、质量门禁与联调顺序
 13. 初始化、部署与上线步骤
 14. V1.5 交付物清单
- 附录 A：状态机
- 附录 B：原型总览（PC + 移动端）

1. 本版目标与冻结范围

V1.5 的定位不是继续扩充概念，而是把 V1.4 工程骨架补成可直接建仓、分工、联调和持续迭代的“完整开发基线”。从本版开始，核心实体关系、状态机、权限码、并发写入规则、响应式适配原则和后端技术路线原则上进入冻结。

- 独立账号与 RBAC：用户、角色、权限、数据范围均由本系统维护。
- 完整核心域：反馈、正式需求、版本、发布记录、通知、评论、附件、审计。
- 正式采用 Python FastAPI；业务规则放 Service 层，Controller 不直接拼状态机。
- 数据库以 PostgreSQL 为优先方案，所有关键对象使用 revision 做乐观锁。
- Redis 用于编辑提示、会话/限流、轻量状态；附件对象存储统一由 StorageService 管理，本地默认 LocalFileStorage，可选 S3Storage。
- 前端采用同一套 Vue 3 工程同时适配桌面和移动端。

范围	一期结论
必须交付	登录、反馈、需求、版本、发布、通知、用户角色权限、系统模块字典、附件、审计
移动端	核心业务全部可查看；提交反馈、查看进度、轻量处理必须可用
并发编辑	任何关键写入必须校验 revision；过期写入返回 HTTP 409
发布规则	只有 READY 版本可发布；发布同步 DONE 需求及其反馈上线状态
暂不纳入	复杂实时协同编辑、Sprint/燃尽图、工时、完整测试用例平台、多租户 SaaS

2. 最终技术架构与部署拓扑



一期推荐 Docker Compose 部署，先保证可复制、可恢复、可运维；后续并发量和组织复杂度上升后再迁移 Kubernetes，不提前增加部署复杂度。

层次	选型	职责
Web	Vue 3 + TS + Vite + Element Plus	页面、路由、响应式布局、展示层权限
API	FastAPI + Pydantic v2	REST 接口、参数校验、鉴权入口
Domain	Service + 状态机	事务、状态转换、业务约束、审计事件
Persistence	SQLAlchemy 2 + Alembic	ORM、Repository、数据库迁移
Database	PostgreSQL	核心业务、关系、审计、通知
Cache	Redis	编辑提示、登录限流、轻量状态
File	LocalFileStorage / S3Storage	附件二进制；DB 只保存元数据

3. 项目仓库与模块边界

V1.5 已按 backend / frontend / deploy / spec 四部分整理项目仓库。建议后续所有开发 Agent 都在同一仓库基线继续增量开发，不再各自重建结构。

```

rvms_v1_5/
├─ backend/
│   ├── app/api/v1/      # auth / feedback / requirement / version / release / system
│   ├── app/core/        # config / database / security / exceptions
│   ├── app/models/      # SQLAlchemy entities + enums
│   ├── app/schemas/    # Pydantic request/response
│   ├── app/repositories/ # 数据访问
│   ├── app/services/    # 事务、状态机、审计、并发规则
│   ├── app/cli/seed.py  # 初始化管理员/权限
│   └─ alembic/          # migration
├─ frontend/
│   ├── src/api/         # API 封装
│   ├── src/views/       # 页面
│   ├── src/components/  # 自适应与通用组件
│   ├── src/stores/      # Pinia
│   └─ src/router/       # 路由
├─ deploy/docker-compose.yml
└─ spec/openapi-v1.5.yaml

```

后端模块	一期职责	关键权限
Auth	登录、刷新 Token、会话入口	公开 / 登录态
Feedback	提交、查看、编辑、转需求	rd.feedback.*
Requirement	直接创建、编辑、状态、版本迁移	rd.requirement.*
Version	创建、编辑、状态、发布	rd.version.*
Release	历史发布记录	rd.release.view
Notification	站内消息、已读	登录用户本人
User / Role	账号、角色、权限绑定	sys.user.* / sys.role.*

System	业务系统/模块字典	sys.system.*
File	上传附件与元数据	登录用户 + 业务权限
Audit	关键操作审计	sys.audit.view

4. 后端完整开发基线

4.1 分层规则

- API 层：解析参数、挂载 Depends、返回 DTO；不直接修改多张业务表。
- Service 层：状态机、事务、跨表更新、审计、通知事件。
- Repository 层：查询、分页、数据范围过滤、revision 条件更新。
- Model 层：数据库实体；Schema 层：请求/响应模型。
- 所有核心写操作都必须通过 Service，不允许在路由中绕过状态机。

4.2 Python / FastAPI 版本基线

组件	建议版本/方式
Python	3.12+
FastAPI	0.116+
Pydantic	v2
SQLAlchemy	2.x
Alembic	1.16+
PostgreSQL Driver	psycopg 3
Password	Argon2id (pwdlib)
JWT	PyJWT / HS256 (一期)
Redis Client	redis-py
Object Storage	boto3 S3 API

4.3 关键后端实现已经进入骨架

- AuthService：用户名密码认证、Access/Refresh Token。
- FeedbackService：提交、revision 编辑、反馈转正式需求事务。
- RequirementService：直接创建、编辑、状态机、跨版本迁移。
- VersionService：版本创建/状态流转/发布事务，发布后同步需求与反馈上线。
- EditingService：Redis 维护“正在编辑”提示，不做强排他锁。
- AuditService：关键实体变化写 OperationLog。

5. 数据库与 Alembic 迁移基线

核心数据采用 PostgreSQL。V1.5 项目骨架已包含 Alembic 初始化 migration，后续任何表结构变化必须新增 migration，不允许直接在线上手改表结构。

表	作用	关键约束
sys_user	独立用户账号	username 唯一，状态 ACTIVE/DISABLED/LOCKED
sys_role / sys_permission	RBAC	role.code、permission.code 唯一
sys_user_role / sys_role_permission	角色关系	联合主键
sys_business_system / module	业务系统与模块	module(system_id, code) 唯一
rd_feedback	原始反馈	feedback_no 唯一；revision
rd_requirement	正式需求	requirement_no 唯一；revision
rd_requirement_feedback	反馈与需求	一条反馈最多一个主需求
rd_version	版本容器	version_no 唯一；revision
rd_version_requirement	版本需求关系	同一需求同时仅一条 active 关系
rd_release	发布记录	每次实际发布留痕
sys_notification	站内通知	user_id + read_at
sys_file / attachment_relation	附件	二进制不入业务表
rd_comment	讨论记录	entity_type + entity_id
sys_operation_log	操作审计	不可由普通用户修改

5.1 并发控制字段

```
revision INTEGER NOT NULL DEFAULT 1
updated_at TIMESTAMPTZ NOT NULL
updated_by BIGINT NULL
```

所有来自 PC 和移动端的关键编辑请求，都必须提交“页面读取时拿到的 revision”。保存成功后 revision + 1；旧 revision 更新影响行数为 0 时统一返回 409。反馈转需求、需求迁版和版本发布等跨表事务也必须先执行带 revision 条件的原子更新。

6. API 契约与权限约束

V1.5 单独提供 OpenAPI 3.1 文件。API 基础前缀统一为 /api/v1，除登录/刷新外均要求 Bearer JWT。关键写接口必须进行后端权限校验。

Method	Path	用途	权限/规则
POST	/auth/login	登录	公开
POST	/auth/refresh	刷新 Token	Refresh Token
GET/POST	/feedbacks	反馈列表/提交	rd.feedback.view/create

PATCH	/feedbacks/{id}	编辑反馈	revision + rd.feedback.edit
POST	/feedbacks/{id}/convert	反馈转需求	事务 + rd.feedback.convert
GET/POST	/requirements	需求列表/新建	rd.requirement.view/create
PATCH	/requirements/{id}	编辑需求	revision + rd.requirement.edit
PATCH	/requirements/{id}/status	状态流转	状态机 + revision
POST	/requirements/{id}/move-version	调整版本	reason 必填 + revision
GET/POST	/versions	版本列表/创建	rd.version.view/create
POST	/versions/{id}/publish	版本发布	READY + revision + 成功发布事务 (固定 SUCCESS)
GET	/releases	发布记录	rd.release.view
GET	/notifications	通知中心	本人数据
POST	/editing/*	编辑提示	登录态
GET/POST	/users	用户管理	sys.user.*
GET/POST	/roles	角色权限	sys.role.*
POST	/files	上传附件	MIME 白名单 + 50MB

7. Feedback → Requirement → Version → Release 主链路

7.1 反馈转需求

1. 产品负责人打开反馈详情并点击“转为需求”。
2. 系统预填标题、类型、系统、模块、描述和附件引用。
3. 保存时同一事务创建 Requirement、RequirementFeedback，回填 Feedback.main_requirement_id。
4. 若同时选择版本，则创建 VersionRequirement，并将正式需求设置为 PLANNED。
5. 写入两条审计：Feedback CONVERT_REQUIREMENT、Requirement CREATE_FROM_FEEDBACK。

7.2 版本内 A / B / C 场景

V1.0.1 中 A、B 为直接新建需求，C 来自反馈转化。三者在本版本内完全同级，仅 source 和关联反馈不同。版本永远关联 Requirement，不直接关联 Feedback。

版本项	来源	追溯
A 需求	DIRECT	无原始反馈
B 需求	DIRECT	无原始反馈
C 需求	FEEDBACK	FB → REQ → V1.0.1

7.3 发布事务

READY Version

```
→ create Release
→ Version.status = RELEASED
→ DONE Requirements = ONLINE
→ linked Feedbacks = ONLINE
→ create Notifications
→ Audit Log
→ COMMIT
```

通知发送可在事务提交后异步执行，但数据库状态更新必须保持一致性。V1.5 MVP 只支持成功发布，publish 请求不接收 result；事务固定写入 Release.result = SUCCESS。Version.RELEASED 与 Requirement.ONLINE 只能由该成功发布事务产生。

8. 多人编辑与并发冲突

- 不做 Google Docs 式实时协同；第一期采用“编辑提示 + 乐观锁”。
- Redis key：edit_lock:{entity_type}:{entity_id}，TTL 默认 10 分钟，前端每 2 分钟心跳续期。
- 若他人正在编辑，前端只提示，不禁止进入；最终正确性由 revision 保证。
- HTTP 409 的前端统一弹窗：查看最新内容、对比差异、放弃当前修改、基于最新版继续编辑。
- 状态、优先级、负责人、版本迁移优先拆为独立接口，降低整页覆盖风险。Version 允许 READY → TESTING，Requirement 允许在所属版本尚未发布时 DONE → DEVELOPING；两种退回都必须填写原因并写审计日志。

UPDATE rd_requirement

SET title = :title, revision = revision + 1

WHERE id = :id AND revision = :client_revision;

rowcount = 0 => HTTP 409 Conflict

场景	系统行为
张三、李四同时打开 REQ-0021	两人读取到 revision=6；Redis 可提示“有人正在编辑”
张三先保存	数据库 revision 变为 7
李四后保存 revision=6	后端拒绝更新，返回 409 和当前 revision=7
李四确认最新版	重新加载后基于 revision=7 继续编辑

9. 前端工程与 PC / Mobile 响应式实现

V1.5 继续采用一个 Vue 3 工程覆盖桌面、平板和手机，不建设第二套移动端项目。

场景	PC	Mobile
主导航	固定左侧菜单	底部导航 + 必要时抽屉
列表	Element Plus Table	卡片列表
筛选	顶部多条件栏	筛选抽屉
详情	两栏/多栏信息块	单列堆叠
表单	Dialog/Drawer/页面	独立页面或全屏抽屉
Tabs	常规 Tabs	允许横向滚动
关键操作	工具栏按钮	底部安全区固定按钮
系统设置	完整支持	可提示建议使用 PC

9.1 已纳入 V1.5 的页面

- 登录页
- 首页数据概览
- 反馈列表 / 提交 / 详情
- 需求列表 / 详情
- 版本列表 / 详情
- 发布记录
- 通知中心
- 用户管理
- 角色权限。

9.2 移动端必须验收

- 360px - 430px 宽度下无横向溢出。
- 反馈、需求、版本列表使用卡片，不缩放桌面表格。
- 提交反馈全流程可在单手滚动场景完成。
- 详情页操作按钮不能被浏览器底部安全区遮挡。
- PC 与手机同一账号、同一权限、同一数据、同一 revision 规则。

10. 登录、RBAC 与系统管理

角色建议	典型权限
普通成员	提交反馈、查看本人/可见反馈、查看需求和版本
产品/项目负责人	受理反馈、转需求、创建/编辑需求、版本排期

研发负责人	需求状态、负责人、版本状态
测试	测试状态、版本测试阶段信息
管理员	用户、角色、权限、系统模块字典、审计

权限判定必须以后端为准。Vue route.meta、按钮隐藏仅用于改善体验，不能作为安全边界。

- Access Token 短有效期；Refresh Token 长有效期并预留可撤销会话。
- 密码使用 Argon2id；初始化管理员首次登录强制改密。
- 数据范围预留 SELF / TEAM / ALL。第一期默认以角色范围过滤列表。
- 超级管理员只通过显式“*”或全量权限码获得能力，不用前端硬编码用户名。

11. 附件、通知、审计与安全

11.1 附件

- 业务表只存 file_id 关系，不保存二进制。
- 默认允许 PNG/JPEG/WebP/PDF/TXT/DOCX/XLSX；单文件默认 50MB。
- 上传时验证 MIME + 扩展名并计算 SHA-256。
- 生产环境通过签名 URL 或受控下载接口访问对象存储。

11.2 通知

- 反馈被受理/转需求
- 需求加入或调整版本
- 需求上线
- 版本发布
- @ 评论（后续增强）

11.3 审计

- 登录成功/失败、密码重置
- 反馈转需求、关闭/重开
- 需求状态/优先级/负责人/版本变化
- 版本清单和发布
- 用户角色权限变化

OperationLog 应记录 entity_type、entity_id、action、operator_id、before_data、after_data、request_id、IP、created_at。日志不得包含明文密码、Token 或对象存储密钥。

12. 测试、质量门禁与联调顺序

门禁	最低要求
Python	compileall 通过；pytest 状态机/服务测试通过
API	OpenAPI YAML 可解析；核心接口有 200/401/403/409/422 场景
DB	Alembic upgrade head / downgrade 可验证；约束生效
前端	vue-tsc + vite build 通过；360/768/1200 三断点人工验收
安全	无权限调用返回 403；过期 Token 401；旧 revision 409
业务	Feedback→Requirement→Version→Release 主链路冒烟通过
备份	PostgreSQL 恢复演练至少一次

12.1 推荐联调顺序

1. Auth + 用户/角色/权限
2. 系统/模块字典
3. Feedback
4. Requirement
5. Feedback → Requirement
6. Version + 版本迁移
7. Release 发布事务
8. Notification + Audit
9. 附件
10. PC/Mobile 响应式联调

13. 初始化、部署与上线步骤

1. 复制 .env.example 为 .env，并填写 DATABASE_URL、REDIS_URL、JWT_SECRET、JWT_ACCESS_TTL_MINUTES、JWT_REFRESH_TTL_DAYS、STORAGE_DRIVER、

LOCAL_STORAGE_PATH、S3_ENDPOINT、S3_ACCESS_KEY、S3_SECRET_KEY、
S3_BUCKET、S3_REGION、INIT_ADMIN_USERNAME、INIT_ADMIN_PASSWORD。

2. Docker Compose 启动 PostgreSQL、Redis；本地附件默认写入 LocalFileStorage，不要求启动 S3 服务。
3. 执行 Alembic：alembic upgrade head。
4. 执行 python -m app.cli.seed，写入默认权限码、超级管理员角色和初始管理员。
5. 首次管理员登录强制修改密码。
6. 配置至少一个业务系统及模块。
7. 部署 API 与 Web，开启 HTTPS。
8. 执行反馈转需求、版本迁移、发布、revision 409、移动端四条冒烟链路。
9. 启用数据库每日备份和对象存储备份/版本化。

```
docker compose -f deploy/docker-compose.yml up -d
cd backend && alembic upgrade head
python -m app.cli.seed
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

14. V1.5 交付物清单

交付物	内容
V1.5 完整开发基线 ZIP	FastAPI + Vue 3 + Docker + Alembic + Spec 项目目录
V1.5 OpenAPI 3.1	26 组路由定义，含核心请求模型、JWT、409 并发冲突说明
V1.5 Word / PDF	本开发基线说明文档
Alembic 初始迁移	核心业务表、RBAC、关系表和并发唯一索引
Seed 初始化脚本	默认权限码、超级管理员角色、初始化管理员
PC / Mobile 原型	独立登录、核心 PC 页面与移动端页面方向

开发 Agent 应优先以 ZIP 内代码与 OpenAPI 作为“机器可执行基线”，以本说明文档作为业务和工程规则解释。若二者发生冲突，应先停止开发并修订同一版本基线，不允许各自猜测。

附录 A：状态机

Requirement

DRAFT → CONFIRMED → PLANNED → DEVELOPING → TESTING → DONE

DONE → ONLINE (仅成功发布事务)

DONE → DEVELOPING (版本未发布 + reason + audit)

↘ PAUSED / CANCELED (按规则)

Version

PLANNING → DEVELOPING → TESTING → READY

READY → RELEASED (仅 publish)

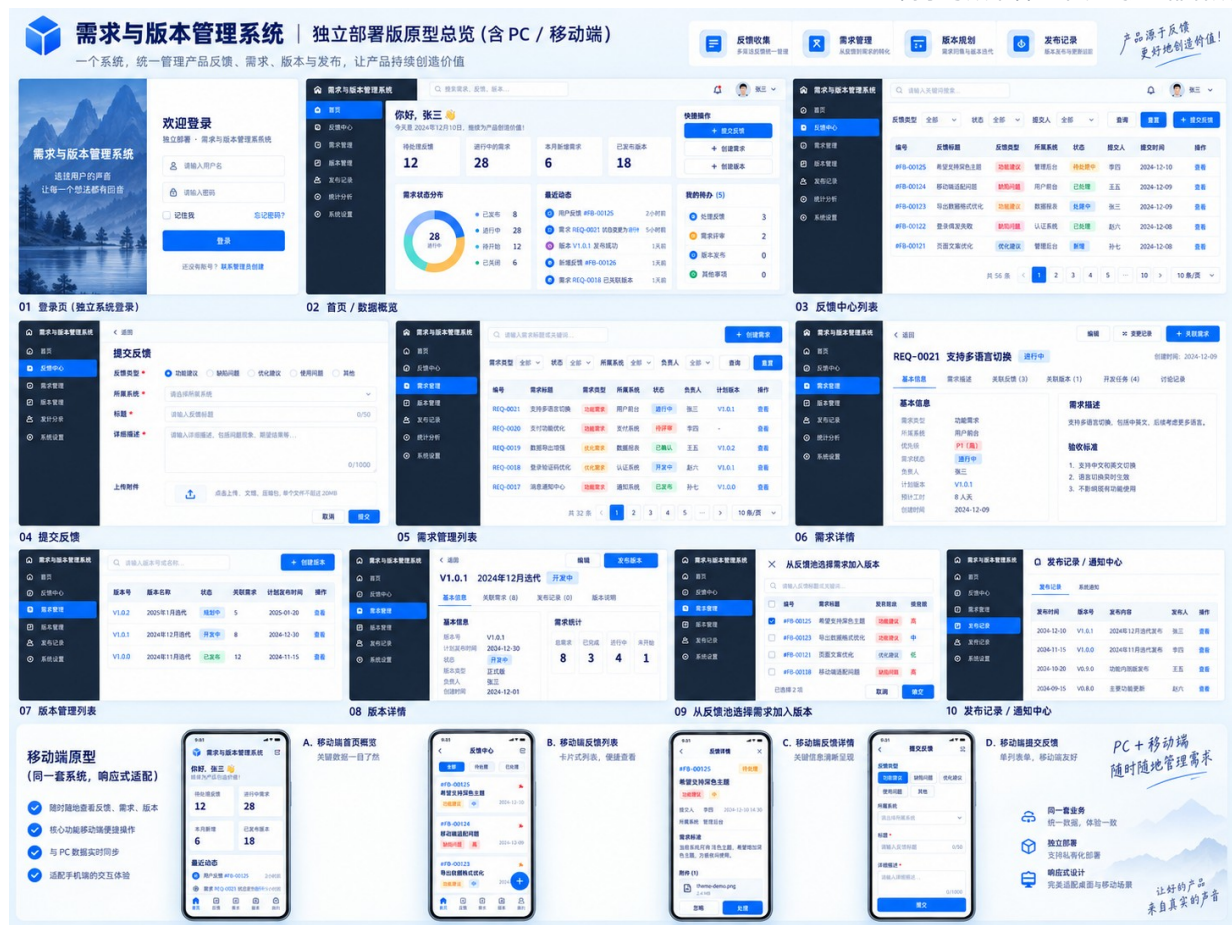
READY → TESTING (reason + audit)

↘ CANCELED

实体	终态	重要规则
Feedback	ONLINE / CLOSED 等	转需求后保留原文和来源；上线后仍可追溯
Requirement	ONLINE / CANCELED	版本内所有项同级；来源不改变需求层级
Version	RELEASED / CANCELED	RELEASED 原则上不修改需求清单
Release	记录型实体	一次实际发布对应一条 Release

附录 B：原型总览 (PC + 移动端)

以下原型继续作为独立部署版的视觉与信息架构基线。移动端采用同一业务与接口，仅改变布局、导航和交互承载方式。



原型覆盖：独立登录、首页、反馈中心、提交反馈、需求列表/详情、版本列表/详情、发布/通知，以及移动端首页、反馈列表、反馈详情、提交反馈等核心场景。